

Plan de Desarrollo Técnico: Agente Knowledge Concierge (RAG)

Este documento detalla la estrategia de implementación para el agente encargado de resolver dudas frecuentes (precios, servicios, políticas) dentro del ecosistema del "Agente Autónomo de Ventas y Reservas"¹¹¹¹.

1. Arquitectura RAG Recomendada: "Hybrid RAG con Re-ranking"

Dado que el dominio es médico/dental y transaccional, la precisión es crítica. Un modelo puramente semántico (vectorial) puede fallar al distinguir entre "precio de implante" y "precio de limpieza" si los textos son similares. Por ello, el mejor enfoque es una **Búsqueda Híbrida**.

¿Por qué Hybrid RAG?

- **Búsqueda Vectorial (Semántica):** Entiende el significado. Ejemplo: El usuario pregunta "¿Me va a doler?" y el sistema recupera información sobre "Anestesia y procedimientos sin dolor", aunque no compartan palabras clave.
- **Búsqueda por Palabras Clave (Léxica - BM25):** Garantiza exactitud en términos específicos. Ejemplo: Si el usuario busca "Ortodoncia Invisible", el sistema prioriza documentos que contengan ese término exacto, evitando confundirlo con "Ortodoncia metálica".

Flujo de Recuperación (Retrieval Workflow)

1. **Query Expansion:** El **Orchestrator** envía la pregunta². Un LLM ligero reescribe la consulta para mejorar la búsqueda (ej: de "¿cuánto duele?" a "niveles de dolor en tratamientos dentales").
2. **Dual Retrieval:** Se ejecutan dos búsquedas en paralelo contra la base de datos:
 - Búsqueda de vectores densos (Embeddings).
 - Búsqueda de palabras clave (Sparse vectors/BM25).
3. **Reciprocal Rank Fusion (RRF):** Se combinan los resultados de ambas búsquedas para obtener una lista única de candidatos.
4. **Re-ranking (Cross-Encoder):** Un modelo especializado (como Cohere Rerank o BGE-Reranker) reordena los 5-10 mejores resultados basándose en qué tan bien responden realmente a la pregunta. Esto es crucial para asegurar que la respuesta "verificada"³ sea la correcta.

2. Selección de Base de Datos Vectorial (Vector DB)

Recomendación: Qdrant o Pinecone

La elección ganadora: Qdrant.

- **¿Por qué?** Qdrant soporta nativamente la **Búsqueda Híbrida** (Vectores + Lexical/BM25) en una sola consulta, lo cual simplifica enormemente la arquitectura. Además, tiene un filtrado de metadatos extremadamente rápido.
- **Contexto de Negocio:** Como el sistema puede servir a múltiples "profesionales médicos"⁴, necesitarás filtrar la información por `clinic_id` o `doctor_id`. Qdrant maneja este *multi-tenancy* eficientemente.

3. Estrategia de Vectorización (Embeddings)

¿Cómo vectorizar?

No se debe vectorizar el documento entero como un solo bloque. Se debe utilizar una estrategia de **Chunking Semántico**.

1. **Modelo de Embeddings:** Recomiendo **text-embedding-3-large** de OpenAI o **multilingual-e5-large**.
 - *Justificación:* Tienen excelente desempeño en español⁵ y una ventana de contexto amplia.
2. **Proceso de Similitud:**
 - Se utiliza **Cosine Similarity** (Similitud del Coseno). Mide el ángulo entre el vector de la "pregunta del usuario" y los vectores de los "fragmentos de información".
 - Matemáticamente, si el ángulo es cercano a 0 (coseno cercano a 1), significa que semánticamente hablan de lo mismo.

4. Estructura de Datos y "Chunking"

La calidad de la respuesta del **Knowledge Concierge** depende 100% de cómo se estructure la información antes de guardarla.

¿Qué datos debe tener el agente?

El agente necesita acceso estructurado a:

1. **Catálogo de Servicios:** Nombre, Descripción técnica, Duración, Precio base, Requisitos previos.
2. **Políticas Comerciales:** Cancelación, Reembolsos, Métodos de pago, Seguros aceptados.
3. **Información Logística:** Dirección, Mapa (link), Horarios de atención, Estacionamiento.

4. **Preguntas Frecuentes (FAQ):** "¿Duele?", "¿Aceptan niños?", "¿Tienen sedación?".

Diseño del "Chunk" (La unidad de información)

En lugar de guardar texto plano, guardaremos objetos JSON enriquecidos con metadatos para facilitar el filtrado.

Ejemplo de cómo se ve la información vectorizada:

Tipo A: Servicio Médico

JSON

```
{
  "id": "srv_001",
  "text": "La endodoncia (tratamiento de conducto) tiene un costo base de $200 USD. Incluye la curación, anestesia local y radiografía periapical inicial. La duración promedio es de 60 a 90 minutos por sesión. Es necesaria cuando hay dolor agudo o infección profunda.",
  "metadata": {
    "category": "Servicios",
    "subcategory": "Endodoncia",
    "price": 200,
    "currency": "USD",
    "clinic_id": "dental_asia_01",
    "source": "Lista_Precios_2025_v2.pdf"
  },
  "vectors": [0.012, -0.045, ... ] // Vector generado por el modelo
}
```

Tipo B: Política de Cancelación

JSON

```
{
  "id": "pol_005",
  "text": "Para cancelar o reprogramar una cita sin penalidad, el paciente debe notificar con al menos 24 horas de anticipación. Si la cancelación ocurre con menos tiempo, se cobrará una penalidad del 20% del valor de la consulta reservada o se perderá el depósito.",
  "metadata": {
    "category": "Políticas",
    "topic": "Cancelación",
    "strictness": "High",
    "clinic_id": "dental_asia_01",
    "source": "Terminos_y_Condiciones.pdf"
  },
  "vectors": [-0.002, 0.123, ... ]
}
```

¿Por qué esta estructura?

Cuando el usuario pregunte: "*¿Puedo cambiar mi cita de mañana?*", el sistema buscará vectores similares a "cambiar cita" PERO el **Orchestrator** o el filtro de búsqueda pueden restringir la búsqueda a **category: "Políticas"**, asegurando que no se recupere información irrelevante sobre precios.

5. Roadmap de Implementación

Fase 1: Ingesta y Procesamiento (ETL)

1. **Recolección:** Conectar con el CRM y documentos internos (PDFs de precios, Docs de TyC)⁷⁷⁷⁷.
2. **Limpieza:** Eliminar encabezados, pies de página y formato irrelevante.
3. **Chunking:** Dividir los documentos.
 - *Estrategia:* Usar **MarkdownSplitter** (si los docs tienen títulos) o **RecursiveCharacterTextSplitter**. Tamaño sugerido: 500 caracteres con 50 de solapamiento.
4. **Enriquecimiento:** Usar un LLM para generar una "pregunta sintética" para cada chunk. (Ej: Al chunk de cancelación, agregarle la pregunta oculta "¿Qué pasa si no voy a mi cita?"). Esto mejora drásticamente la búsqueda.
5. **Indexación:** Subir vectores y payloads (metadatos) a Qdrant.

Fase 2: Construcción del Pipeline RAG

1. Configurar la conexión con OpenAI API (para generación y embeddings).
2. Implementar la lógica de **Hybrid Search** (peso sugerido: 0.7 Vectorial / 0.3 Keyword).
3. Diseñar el **Prompt del Sistema** para el Knowledge Concierge:
"Eres un asistente útil de una clínica dental. Usa SOLO el contexto proporcionado para responder. Si la respuesta no está en el contexto, di 'No tengo esa información verificada' y sugiere contactar a un humano. No inventes precios."

Fase 3: Integración con Orchestrator y Guardian

1. **Input:** El Knowledge Concierge recibe {pregunta, contexto} del Orchestrator⁸.
2. **Output:** Genera {respuesta verificada, fuente, confianza}⁹.
3. **Validación:** El Guardian verifica que el precio mencionado en la respuesta coincida con el precio en el chunk recuperado¹⁰.

Fase 4: Evaluación

Utilizar el framework **RAGAS** para medir automáticamente:

- **Faithfulness:** ¿La respuesta se basa únicamente en el contexto recuperado? (Evitar alucinaciones de precios).

- **Answer Relevance:** ¿La respuesta atiende la duda del usuario?

Resumen de Tecnologías Sugeridas

Componente	Tecnología Recomendada	Por qué
Vector DB	Qdrant (o Pinecone)	Soporte nativo Híbrido y filtrado rápido.
Embeddings	OpenAI text-embedding-3-large	Mejor comprensión semántica en español.
LLM (Generación)	GPT-4o	Razonamiento superior para seguir instrucciones de "no inventar".
Framework	LangChain o LlamaIndex	Facilita la orquestación del RAG y la carga de documentos.
Reranker	Cohere Rerank	Mejora la precisión final antes de generar la respuesta.